

データ仕様

目次

1 正データが壊れたときの多層防御	3
2 人が後から調整するためのフラグ	5
3 data/extract-report.json(監査ログ)	6
4 ハッシュ連鎖の実装	11

i ノート

スキーマの機械的な正は [legacy-reverse/references/schema.md](#)(skill が実行時に参照する原本)。本章はそれに設計意図を加えた解説版。両者が食い違ったら schema.md が正。

データ設計の原則(全体)

- **functions.json が正データ**。⑤の機械抽出が生成・マージする。WBS・骨子・Sphinx 索引は すべてここから再生成される派生物。手書き修正は functions.json に対して行い、派生物は作り直す
- **完了状態は成果物自身が持つ**(フロントマターの status / ハッシュ)。別の進捗 DB を持たない。「ファイルが正しければ進捗も正しい」が常に成り立ち、**再開が自明**になる
- **ledger.json はスクリプト専用の最小限の台帳**(③ freeze ハッシュ・blocked・attempt 基準時刻)。人も LLM も手編集しない
- 機械可読メタデータは YAML フロントマターに集約。WBS・⑥はフロントマター走査で自動生成
- 成果物フロントマターに独自キーを足すときは Quarto 予約キーと衝突させない (前例: coverage → tc-coverage に改名)
- 表示専用の使い捨てデータは **.legacy-reverse/ に隔離**する。進捗の正は常に成果物側にあり、**.legacy-reverse/** を消しても失われるのは実行ログと画面表示だけ

1 正データが壊れたときの多層防御

functions.json は全スクリプトが毎回読む。ここが壊れると無人バッチが途中で止まるため、AI の編集ミス(末尾カンマ等)を 3 段で受け止める。

表 1-1

段	仕組み	効果
1. 発生の直後	hooks/guard_json.py(PostToolUse)が編集直後に構文検証	壊れた位置(行・列)を添えて エージェント自身に差し戻す 。人を待たずにその場で直る
2. 読み込み時	ledger.load_json が行番号付きの明示エラーで停止	生の traceback ではなく「どこが・どう壊れ・どう直すか」を出す
3. 実行系	バッチ・単発・サーバがこれを「データ異常のため停止」として扱う	画面に理由が残る(無言で止まらない)

対象プロジェクトのディレクトリ構成

```
<project>/
  legacy/                                # レガシー原文(読み取り専用。読めるのは ⑥ と ①改訂のみ)
  src/<package>/                          # ④ 新実装
  tests/                                  # ③ テストコード(⑤ループ中は hook が編集拒否)
  docs/
    _quarto.yml                          # HTML サイト+PDF 出力設定(テンプレから⑥で配置)
    wbs.css                              # WBS 関数一覧の列幅(テンプレからコピー)
    index.qmd                             # WBS(ledger.py wbs で自動生成。手編集禁止)
    wbs/                                  # 200 関数超で自動生成されるファイル別明細(手編集禁止)
    spec-review.md                        # ① draft の一斉レビュー表(自動生成。手編集禁止)
    review-feedback.md                   # ブラウザからの修正依頼(サーバが追記・skill が applied 化)
    _site/                                # HTML 出力(render_site.py が作り直す。git 管理外)
    _sitework/                            # render 用の .qmd 影コピー(作業用。残らない)
    conventions.md                       # プロジェクト規約(⑥で確定。人が編集可)
    domain-knowledge.md                  # 裁定の蓄積(人が編集可)
    specs/F-xxxx.md                     # ① 関数仕様書
    test-specs/F-xxxx.md                 # ② テスト仕様書
    test-results/F-xxxx_日時.md          # ⑤ 結果報告書(毎回新規・自動生成)
    issues/ISSUE-xxx.md                  # 質問と裁定(回答欄は人が記入可)
    improvements/XXX-nnn.md              # ⑦ 施策票
    completion-check.md                  # ⑥(自動生成)
    quant.md / perf.md                   # ⑦ 定量計測の結果(quant_analyze.py が生成)
    analysis.md                          # ⑦ 施策候補(AI が実測に基づいて記入)
  docs-sphinx/                           # ④ API ドキュメント設定(テンプレから⑥で配置)
  data/
    functions.json                       # ⑥の解析結果(正データ)
    extract-report.json                  # ⑥機械抽出の監査ログ
    ledger.json                          # ハッシュ・ブロック状態(スクリプト専用)
  .legacy-reverse/                       # 実行時データ(git 管理外・消しても成果物は失われない)
    state.json                           # 実行中フェーズ(hook の判定に使う)
    run.lock                             # 実行スロットの排他(PID 入り。残骸は自動解除)
    pipeline-status.json                 # ライブ進捗(バッチ実行状況ページが読む・表示専用)
    pipeline-log.jsonl                   # 実行ログ 1 行 1 試行(結果・コスト・所要)
    agent-logs/F-xxxx.txt                # エージェント応答の全文(失敗原因の一次情報)
    batch-priority.json                  # ★優先の割り込み順(画面が書く)
    last-run.json                        # pytest プラグインの出力(⑤の一時データ)
    quant.json                           # ⑦の実測データ(機械可読)
```

state.json と run.lock は役割が違う。前者は hook が「いまどのフェーズか」を判定するためのフェーズ状態、後者は 実行の二重起動を防ぐ排他ロック。

data/functions.json

①の機械抽出(extract_fortran.py / extract_c.py)が生成・マージする正データ。言語混在プロジェクトでは複数の抽出器が同じファイルにマージする。

```
{
  "project": {
    "name": "string",
    "legacy_lang": "fortran | csharp | ...",
    "new_lang": "python",
    "package": "newpkg"
  },
  "functions": [
    {
      "func_id": "F-0123",
      "legacy": { "file": "legacy/tax.cbl", "name": "CALC-TAX", "lines": "120-240" },
      "new": {
        "module": "src/newpkg/tax.py",
        "name": "calc_tax",
        "signature": "calc_tax(amount: Decimal, rate_code: str) → Decimal"
      },
      "inputs": [ { "name": "WK-AMOUNT", "legacy_type": "PIC 9(9)V99", "new_type": "Decimal",
"desc": "課税対象額" } ],
      "outputs": [ { "name": "WK-TAX", "legacy_type": "PIC 9(9)V99", "new_type": "Decimal",
"desc": "税額" } ],
      "globals": [ { "name": "TAX-RATE-TABLE", "access": "read", "desc": "税率マスタ" } ],
      "external_files": [ { "path": "RATEMST.dat", "access": "read", "desc": "税率マスタファ
イル" } ],
      "calls": [ "F-0087" ],
      "test_file": "tests/test_tax.py"
    }
  ]
}
```

- func_id は F- + 4 桁連番。採番は抽出器が行い、マージ再実行でも不変
- F-0000 はメインルーチンの予約番号(Fortran の program / C の main)。コールグラフの根なので推奨着手順では最後に回る
- calls は func_id の配列 — コールグラフの正データ(WBS の推奨着手順=葉から、の根拠)
- unresolved_calls(任意)は抽出時に解決できなかった呼び出し名。別言語の抽出が走った時点で自動的に解決されて calls に移るので、抽出の実行順を問わない
- test_file は①では省略可。③が確定させる

2 人が後から調整するためのフラグ

①の抽出漏れ・関数分割・デッドコードは、人の指示で後追い調整する (ledger add / exclude / include、MCP では add_function 等)。

表 2-1

フラグ	意味	設計判断
manual: true	人の指示で後追い追加したエントリ	抽出の再実行で「ソースに無い」警告を出さない。実ソースで確認されたら自動で外れる
excluded: true + excluded_reason	移植対象外	①～⑥・WBS・next の対象から外れ、WBS の「対象外の関数」に理由つきで残る

エントリの物理削除は禁止。抽出を再実行するとソースから別の func_id として復活し、既存の成果物との紐付けが切れるため。対象から外すのは必ずフラグで行う。

3 data/extract-report.json(監査ログ)

⑩の機械抽出のたびに更新される。人と LLM がレビューすべき項目:

表 3-1

項目	意味
completeness_mismatches	2 系統カウント突合の差分(数え漏れの疑い)。ゼロが⑩完了条件
inferred_calls	静的解析で推定した呼び出し関係
unresolved_calls	解決できなかった呼び出し名(外部ライブラリ等の可能性)
マージ差分	再実行時の追加・変更・保持の内訳

data/ledger.json

スクリプト(ledger.py / collect_results.py)だけが読み書きする。手編集禁止。

```
{
  "F-0123": {
    "test_code_hash": "e5f6a7b8",
    "blocked_by": null,
    "attempt_reset_at": "2026-07-25T10:00"
  }
}
```

表 3-2

キー	意味
test_code_hash	③ freeze 時のテストファイル sha256 先頭 8 桁。現物と不一致 = △改変
blocked_by	④⑤ループ上限到達時の ISSUE-ID。人の裁定後 <code>unlock</code> で null に戻る
attempt_reset_at	この時刻以降の結果ファイル数 = attempt(⑤実行回数)。unlock でリセット

.legacy-reverse/state.json

```
{ "phase": "5", "func_id": "F-0123" }
```

- フェーズ skill が開始時に `ledger.py phase-start <n> <func-id>`、終了時に `phase-end` で管理
- `hook(guard_tests.py)`は phase が 4 または 5 のとき `tests/` への Edit/Write を拒否する

.legacy-reverse/pipeline-status.json

実行中の状態。表示専用の使い捨てで、進捗の正ではない(正は成果物側)。書き込みはアトミック(tmp → replace)なので、読み手が壊れた JSON を掴むことはない。

```
{
  "state": "running",
  "mode": "連続実行(ブラウザ)",
  "pid": 12345,
  "total_targets": 62, "done": 34, "failed": 5,
  "cost_usd": 11.83,
  "rate_waited_sec": 340, "wait_until": null,
  "current": { "func_id": "F-0032", "attempt": 1, "started_at": "2026-08-07T06:12:03" },
  "ng_kinds": { "機械レビュー NG": 3, "検証 NG": 2 },
  "recent": [
    { "func_id": "F-0019", "ok": false, "phase": "spec",
      "kind": "機械レビュー NG", "problems": ["SPEC-...: 根拠引用がない"],
      "sec": 233, "cost_usd": 0.66, "attempt": 2, "at": "2026-08-07T06:03:28" }
  ],
  "metrics": { "median_sec": 112, "success_rate": 0.872,
    "eta_sec": 2576, "avg_cost_usd": 0.303 }
}
```

表 3-3

キー	意味
state	running / waiting_rate / stopped / finished / not_running
pid	書いたプロセス。死んでいれば画面側が「異常終了」に読み替える(残骸で永久に実行中に見えるのを防ぐ)
recent[].phase	どの工程だったか(spec / testspec / …)。連続実行は工程横断なので、これが無いと結果行が何の失敗か分からない
recent[].problems	機械レビューの指摘の全文。画面で箇条書きにする(件数だけ出して探しに行かせない)
metrics.median_sec	成功した実行の所要時間の中央値。画面の進捗バーの基準になる

.legacy-reverse/batch-priority.json

画面の「★優先」が書く割り込み指示。スクリプト専用。

```
{ "order": ["F-0033", "F-0021"], "retry": ["F-0021"] }
```

表 3-4

キー	意味	消費
order	実行順の割り込み。この順で走査結果の先頭に並べ替える	残り続ける(★解除まで)
retry	失敗スキップ集合からの解除対象	次の走査で消費されるワンショット

`retry` をワンショットにするのは、毎回解除すると失敗し続ける関数が無限に再試行されて バッチが進まなくなるため。

フロントマターの status 遷移(完了判定の正)

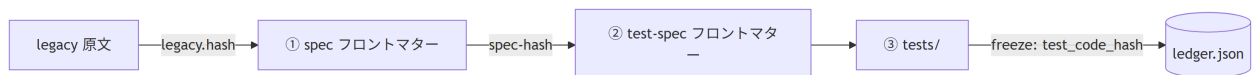
WBS の ☒ 判定・`ledger next` の着手可否判定は、すべてこの表に従う (判定ロジックは `ledger.py` の `status_of()` に一元化)。

表 3-5

成果物	遷移	WBS ☒ 条件
① spec	skeleton → draft → reviewed	reviewed
② test-spec	generated → approved	approved かつ spec-hash が現物と一致
③ test-code	(ファイル+ledger)	ledger の test_code_hash が現物と一致
④ impl	(ファイル)	new.module が存在しスタブ検出ゼロ
⑤ test	(最新結果ファイル)	result: pass(= 実装率 100% かつ失敗 0)
ISSUE	open → answered → applied	— (open は WBS 最上部に露出)
⑦ 施策票	proposed → approved → applied → verified	achieved

4 ハッシュ連鎖の実装

図 4-1



- ハッシュは `ledger.py hash <path>(sha256 先頭 8 桁)`
- `ledger verify <func-id>` が連鎖を検証。不一致 = 上流が改訂された = 下流は「要再生成」
- ⑤の実行前に必ず `verify` が走るため、stale なテストで pass を出すことはできない

WBS の表示記号

表 4-1

記号	意味
✓	完了(上表の条件を満たす)
▲	作業中(draft 等)
□	未着手
⊖	裁定待ち(blocked。ISSUE に回答を)
⚠	stale(上流改訂)または改変(freeze 後変更)→ 要再確認